

Trading Engine - active checklist

Updated **18 September 2026**, following the source review at `6c2f2f6`. This is the single active task list. [Plan v4](#) owns long-range scope; [the guide](#) explains the work and effort estimates.

[TODO.pdf](#) is an export of this checklist. The originally supplied [new-todo-list.pdf](#) is historical input, not the current plan. Documentation edits do not complete the code tasks below. No deadlines are agreed.

Start here

Next learning task: D1, agree the accounting examples; then D2, implement Portfolio. The original foundation repair batch closed on 16 September. ADR 0014 was accepted on 15 September; do not reopen all eight decisions. The review found a narrower D5 accounting gap and additional hardening work, listed separately below.

Recommended impact/effort order: ~~C1 sanitizer enforcement~~ (done 18 September), **D1 accounting rules next**, then C2 capture admission, C3 tape preconditions, then the complete D2-D7 engine path. In-memory Portfolio work can proceed while capture hardening remains open. C2 gates trusted capture-to-engine results; C3 gates tape use.

Fuaad chooses domain rules and writes the first learning-critical implementation. The assistant writes all tests against agreed examples and explains their purpose. Tests must not silently select open accounting or simulation policy.

A. Preserve data and reproducibility

- ✓ **Capture recovery/backup:** user reports the backup task complete. Earlier recovery checks found no remaining `dataless` files. An independent byte comparison of the backup was not performed; do not turn that distinction into a repeated blocker.
- ✓ **Broken build directories:** repaired on 16 September. The recorded cause was iCloud conflict duplicates. Do not repeat the old deletion instructions.
- **Build outside the synced Desktop tree.** Choose and document one configure/build/test workflow, then verify the actual GCC/Clang configurations. **Done when:** the chosen build path is outside sync and a clean build plus mandatory tests pass. Effort: 1-2 hours. Build location remains to be chosen.
- ✓ **Previously pending work:** present in history at the review baseline; the checkout was clean before this documentation update. New changes remain uncommitted until requested.

B. Design status

- ✓ **ADR 0014 D1-D8 accepted, 15 September.** Stage 5 decision inputs are book trust and market shape; operational state belongs to Stage 9.

✓ **ADR 0015 accepted, 16 September:** fatal allocation failure, unsafe object, explicitly interim. This does not mean rollback has been implemented.

- **Complete D5's signed-position accounting policy** through D1 below. The 18 September ADR correction identifies the missing policy; it does not choose it.
- **Choose Strategy dispatch when implementing the seam.** Compare runtime and static alternatives in ADR 0009. Do not assume templates or `std::function` are inherently faster.

C. Additional hardening from the 18 September review

C1. Make sanitizer findings fail CI

✓ **Done 18 September 2026.** `cmake/sanitizers.cmake` now adds `-fno-sanitize-recover=undefined` whenever `TE_SANITIZE` includes `undefined`.

The gap was measured, not assumed. A signed-overflow probe built under the previous configuration printed `runtime error: signed integer overflow` and **exited 0**; with the flag it terminates nonzero before reaching its own `printf`. So UBSan was detecting real UB and letting the run stay green — detection without enforcement.

A compile/link flag rather than `UBSAN_OPTIONS=halt_on_error`: the environment variable binds only where it is exported, so local sanitized builds would have kept recovering silently.

`tests/ub_probe.cpp` holds the deliberate UB. It is guarded by `TE_BUILD_UB_PROBE` (OFF by default), is not registered with CTest, and is never linked into `te_core` or any app, so no intentional UB enters the ordinary suite. A CI step builds it separately per compiler and fails if it exits 0, asserting **nonzero** rather than a specific status, since SIGABRT surfaces differently across platforms.

Scope limit worth remembering: this covers UBSan's checks, which include *signed* overflow. Unsigned wraparound is well-defined in C++, is not UB, and is **not** caught — the explicit boundary tests in `trade_reconciler`, `capture_coordinator` and `price_level` remain the only guard there.

It found a real bug on its first CI run. `Sha256.EmptyInput` failed under both gcc and clang at `sha256.cpp:96:30`: an empty span's `data()` may be `nullptr`, and `memcpy` declares `src` nonnull, so passing it was undefined even with a zero length. Harmless on every real libc, but the compiler may infer from the nonnull attribute that the pointer cannot be null and drop a later check on that basis. That UB predates C1 and UBSan had been reporting it on every sanitized run since the SHA-256 work landed — it just scrolled past in a green log. Fixed in `a1e1f9c`; digests unchanged, all eight known vectors still match.

Note this was a *nonnull-attribute* violation, not signed overflow: local runs were clean because the probe only exercised the signed-overflow check. Do not read "the local suite passed" as "the codebase has no UB."

Verified: 327/327 under UBSan with enforcement active (before and after the fix), 327/327 Release with real `gcc-15` (unaffected, the guard skips non-sanitized builds), and CI green on `build (gcc)`, `build (clang)` and `release` at `a1e1f9c`. The ASan+UBSan suite could not be run locally —

`gtest_discover_tests` fails against ASan-instrumented binaries on this macOS host, a pre-existing limitation; CI covers that config.

C2. Align Python and C++ capture admission

- **Significant; 1-3 days.** Byte/hash/count checks exist. C++ still does not enforce `status` or `chain_valid`. Agree on continuity, ordinals, seed coverage and control boundaries across `manifest_reader`, `segment_loader`, `capture_validator` and the Python validator. **Done when:** shared valid/invalid fixtures receive consistent decisions; interrupted, failed and gapped intervals are rejected or explicitly quarantined under a documented policy. They must never silently become a trusted continuous segment.

C3. Validate tape inputs before opening output

- **Significant; 2-4 hours.** Ordinary replay checks timestamp ordering; the writer does not. Check both streams, seed/cutoff order and header/capture/supplied-seed agreement under the API contract. **Done when:** timestamps [30, 10] with cutoff 20 and inconsistent metadata are rejected before creating output. Do not silently sort a broken stream to conceal a continuity failure.

C4. Preserve useful errors

- **Minor now; 4-8 hours.** Keep detailed loader/validator/replay causes plus segment/location through `captureCoordinator`; format them at the application boundary. **Done when:** a corrupt fixture reports what failed and where without debugging into the loader.

C5. Refresh the supported toolchain contract

- **Minor; 2-4 hours plus compatibility work.** Move local/CI Python from EOL 3.9 to a supported interpreter and run all Python tests. Keep deliberate archive hashes and dependency pins. **Done when:** the documented local and CI environments agree and pass. Add a local websocket integration test before treating mocks as proof of library behavior.

C6. Enforce allocation policy before recovery or bindings

- **Integration gate; hours to 3 days.** Read ADR 0015 before adding a catching engine wrapper, Python binding or recovery loop. Choose an enforced fatal boundary or exception-safe rollback with explicit process behavior. **Done when:** injected failures prove the chosen behavior; no retained book can be reused corrupt. Quantity-overflow tests do not cover allocation failure.

C7. Extend financial guards alongside accounting

- ✓ **Done 19 September 2026.** `src/engine/portfolio.cpp` and `src/engine/risk.cpp` are now in the guard's exact-financial set, alongside the headers that were already there. Declaring a type in an int64 header never stopped the arithmetic in the matching `.cpp` from passing through a `double`, which is precisely where a rounding error would enter.

Listed **before** those files exist, deliberately. The set is matched against files actually walked, so an entry for a missing file is inert until the file appears — which means the guard is in place the day Portfolio is written rather than retrofitted after. Also renamed `isExactFinancialHeader` to `isExactFinancialPath`, since it no longer covers headers only.

Verified both directions: a `double` planted in `src/engine/portfolio.cpp` produces violations and exit 1, while a `double` in `src/research/statistics.cpp` is still allowed, so probabilities and statistics keep floating point. Guard passes on the real tree; 13/13 Python tests pass.

D. Build the Stage 5 engine

D1. Agree the money rules using examples

✓ **Done 19 September 2026. Twelve rules selected and recorded in ADR 0014 D5.**

Worked by hand so far, each row checked against equity and against realized-plus-unrealized: open long, partial close, final close, open short, partial cover, final cover, and a long-to-short reversal selling 5 while holding 2.

Selected: signed `int64` money at a per-instrument scale finer than the quote currency's minor unit; total basis plus quantity with the average derived, never stored; net fee convention; fees always moving basis against the trader; classification by signed position; proportional reversal fee split; position zero if and only if basis zero; realized produced only by closing.

Also selected: money at **8 decimal places** below one quote-currency unit, chosen from the smallest representable amount rather than the ceiling — one satoshi at \$100,000 is `$0.001` and rounds to zero in whole cents. Duplicates keyed on the **venue's execution ID**, ignored with no state change and retained for the whole run. Overflow **refuses the fill** with a named error, using a 128-bit intermediate for `price x quantity` and a candidate-then-commit update so a half-changed account is unrepresentable. Partial closes **subtract what left** rather than rebuilding from a rounded average — rebuilding multiplies the rounding error by the remaining quantity and invents money, while subtracting conserves the total by construction and hands back the position-zero-implies-basis-zero invariant for free.

Short-to-long by buying is deliberately not hand-worked — it is the structural mirror of the reversal already done, and belongs in a test rather than a paper row.

Only the fee *schedule* (flat, basis-point, maker/taker) is left, and it is a D2 interface question, not an accounting rule. These are accepted intent; nothing is test-verified.

D2. Implement Portfolio and its fill journal

- The assistant writes tests from D1; Fuaad writes the first implementation. Use distinct signed exact types and checked wider multiplication before rescaling. Calculate a candidate update before committing state; record execution and fee identities. **Done when:** long/flat/short/reversal cases pass; failed and duplicate fills cannot half-change state; journal replay rebuilds the account exactly. Complete C7. Fill accounting does not obtain a mark price or wall clock.

D3. Define intentions, reasons and the strategy boundary

- Add only required value types and read-only, lifetime-bounded book access. Strategy observation consumes successfully processed inputs; decisions return intentions without execution authority. Do not introduce Stage 9's `OperationalState`. **Done when:** trust/shape blocks have distinct tested reasons and `NoopStrategy` uses the real interface without producing an order. Settle ADR 0009 when the seam needs it.

D4. Build the simulated venue and real admission checks

- Support submit, accept, reject, partial/full fill, cancel request and acknowledgement. Count accepted outstanding orders in worst-case exposure; do not assume opposing orders fill together. Implement quantity, notional and resulting absolute-position limits. **Done when:** transitions are independently testable, rejection cannot mutate venue/portfolio state, and a fill can correctly race a pending cancellation.

D5. Implement scheduling and information availability

- Keep one thread and inject feed/clock/strategy/venue dependencies. Preserve metadata for zero, fixed (80 ms) and recorded availability policies. Specify ties and how delayed/out-of-order observations preserve a valid strategy view. **Done when:** timelines prove apply-before-observe, no future information, market-before-own-arrival ties, submission-sequence ties and arrival-time fill eligibility. Stored receipt time is deterministic input, not a calibrated network-delay measurement.

D6. Pass the complete engine proof

- Wire feed, book, health, strategy, gate, admission, venue and portfolio. Close C2 before trusted capture runs and C6 before exception recovery. **Done when:** no-op conservation, a hand-calculated scripted fill, a real unchanged-state risk rejection and ten repeated runs pass. Save partial-fill, outstanding-exposure, duplicate-fill and cancel-race cases. Compare intermediate event/order/account state, not only final totals.

D7. Ship the replay executable and run record

- Connect `apps/replay_main.cpp` to the build and actual engine. **Done when:** one command runs a mandatory fixture without private data and reports counts, trust, shape, blocks/rejections, orders, accounting and fingerprints. Record input hashes, commit/build and dirty state, scales, policies, configuration, exclusions and result hashes. Detailed traces are optional. Refresh this checklist and handoff with actual build/test evidence.

E. After Stage 5: learning and research gates

Planned, not implemented; detailed requirements live in plan v4.

- **L3 evidence before Stage 6/8 claims:** compare order IDs/quantities at intermediate checkpoints, and priority only where known. Keep the L2 digest for depth checks. Effort: 1-3 days.

- **Tape equivalence before replacing raw replay:** resolve pre-seed classifier warm-up, bind source lineage and preserve or explicitly reject unavailable timing policies. Prove raw/tape equivalence for each supported policy. Effort: 2-4 days; format work may add time.
- **Stage 6:** observed-order labels, explicit cancellation/ambiguity/censoring, availability-safe features and transparent baselines.
- **Stage 7:** chronological sessions, label-interval leakage protection, frozen evaluation, calibration, block uncertainty and sensitivity to assumptions.
- **Stage 8:** equivalent optimized book, controlled profiles and allocation/cache measurements. Compare bounded SPSC with a mutex queue after the single-thread baseline. Add relevant fuzzing, fault injection and concurrency checks.
- **Stage 9:** read-only live feed, recovery, paper lifecycle, operational risk, bounded telemetry and shutdown/failure runbooks. No real-money trading.
- **Stage 10:** thin dashboard, ownership-safe C++/Python boundary, reproducible correctness/performance/research reports and a five-minute demonstration.

Advanced techniques may be contained learning experiments. Promotion to the main engine requires correctness equivalence and measured value. A measured non-improvement still teaches something.

F. Completed foundation evidence

Historical observations, not new test results from this documentation update:

- ✓ Exact types/parsing, clock seam, byte codecs and portable v3 segment I/O.
- ✓ Move-only reference book, ordinary-error atomicity and trust/shape separation.
- ✓ Joined capture, merge/reconciliation, mandatory synthetic fixture and input accounting.
- ✓ Reported joined checkpoint agreement: 0 of 4,533 price levels differed. This does not prove every order's identity or priority.
- ✓ Coordinator byte/hash/count admission, overflow guards and optional-corpus skip repair.
- ✓ All four Python test files in CI, hashed C++ archives, GCC/Clang Debug sanitizer builds, a Release job and explicit debug-only structural validation.
- ✓ ADRs 0014/0015 and prior build-directory repair. Historical logs report 325-case toolchain runs and a later 327-case local run on 16 September; cite configuration/date.

[The guide](#) explains the work and conditional timeframe. [The handoff](#) records current source limitations.